
Convex Polyhedra as an Abstract Domain: An Informal Introduction

R. BAGNARA

Dipartimento di Scienze Matematiche, Fisiche e
Informatiche
Università di Parma

`roberto.bagnara@unipr.it`

THE ABSTRACT INTERPRETATION FRAMEWORK

An instance of [Cousot and Cousot, JLC '92, Section 7].

- The **concrete domain** $\langle C, \sqsubseteq, \perp, \top, \sqcup, \sqcap \rangle$ is a complete lattice;
- The concrete approximation relation $c_1 \sqsubseteq c_2$ holds if c_1 is a stronger property than c_2 ;
- The concrete semantics is $c = \mathcal{F}^\omega(\perp)$, where $\mathcal{F}: C \rightarrow C$ is continuous.

THE ABSTRACT INTERPRETATION FRAMEWORK

An instance of [Cousot and Cousot, JLC '92, Section 7].

- The **concrete domain** $\langle C, \sqsubseteq, \perp, \top, \sqcup, \sqcap \rangle$ is a complete lattice;
- The concrete approximation relation $c_1 \sqsubseteq c_2$ holds if c_1 is a stronger property than c_2 ;
- The concrete semantics is $c = \mathcal{F}^\omega(\perp)$, where $\mathcal{F}: C \rightarrow C$ is continuous.
- The **abstract domain** $\langle D, \vdash, \mathbf{0}, \oplus \rangle$ is a join-semilattice;
- The two domains are related by a monotonic and injective concretization function $\gamma: D \rightarrow C$; thus, the abstract partial order $\vdash$ is indeed the approximation relation induced on D by γ .
- We assume the existence of a sound monotonic abstract semantic function $\mathcal{F}^\sharp: D \rightarrow D$, so that

$$\forall c \in C : \forall d \in D : c \sqsubseteq \gamma(d) \implies \mathcal{F}(c) \sqsubseteq \gamma(\mathcal{F}^\sharp(d)).$$

PROGRAM ANALYSIS/VERIFICATION AND POLYHEDRAL COMPUTATION

- Polyhedral computation meets program analysis and verification for the realization of **numerical abstract domains**.
- These, in turn, can be seen as **domains of approximate assertions**.
- Polyhedral computation is needed to perform **sound inferences over these domains**:
 - as the **complexity/precision tradeoff** is particularly acute in this field, different classes of polyhedra are required to face different situations;
 - **conservative approximation** preserves soundness (but may prevent the derivation of the properties of interest);
 - polyhedra need to be **combined with other numerical abstractions** in order to express useful properties;
 - some non-polyhedral domains can be **encoded** into polyhedra.

A WORKING EXAMPLE (I)

- A collecting semantics gathering relational information about the possible values of numerical variables can be based on the concrete domain:

$$\langle \wp(\mathbb{R}^n), \subseteq, \emptyset, \mathbb{R}^n, \cup, \cap \rangle.$$

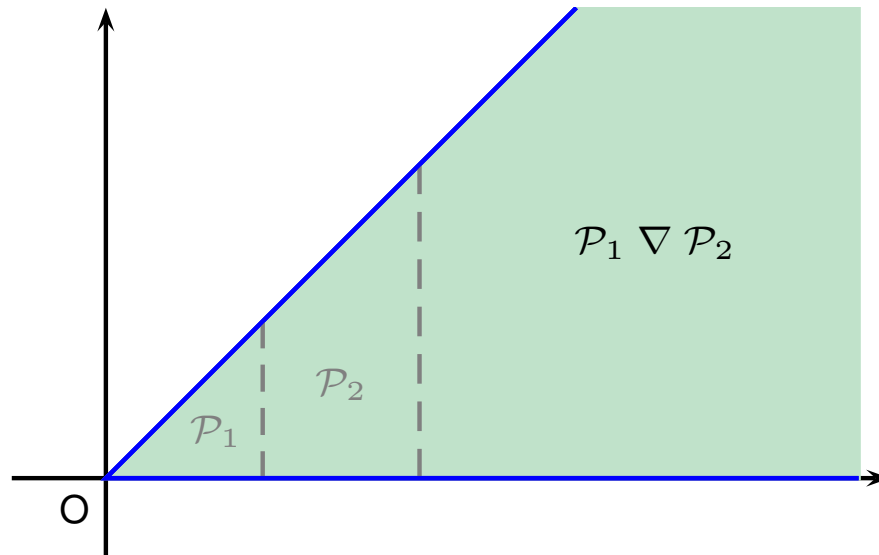
- The abstract domain of closed convex polyhedra [Cousot and Halbwachs, POPL'78] is the (non-complete) lattice

$$\widehat{\mathbb{CP}}_n \stackrel{\text{def}}{=} \langle \mathbb{CP}_n, \subseteq, \emptyset, \mathbb{R}^n, \uplus, \cap \rangle$$

which is related to the concrete domain by $\gamma(\mathcal{P}) \stackrel{\text{def}}{=} \mathcal{P}$.

A WORKING EXAMPLE (II)

- The abstract domain $\widehat{\mathbb{CP}}_n$ has **infinite ascending chains**;
- It comes equipped with the standard widening [Cousot and Halbwachs, POPL'78] or other widenings improving upon it [Bagnara et al., SAS'03].



THE DOMAIN $\mathbb{CP}_n$ OF CLOSED CONVEX POLYHEDRA

A lattice $\langle \mathbb{CP}_n, \subseteq, \emptyset, \mathbb{R}^n, \uplus, \cap \rangle$, with **infinite chains**.

Constraint Representation: $\mathcal{P} = \text{con}(\mathcal{C})$

- $\mathcal{C}$ is a finite set of **linear non-strict inequality** (resp., **equality**) constraints.
- No redundant constraint + max number of equalities $\implies$ **minimal form**.
- Weak notions of **canonical form** (e.g., by orthogonality).

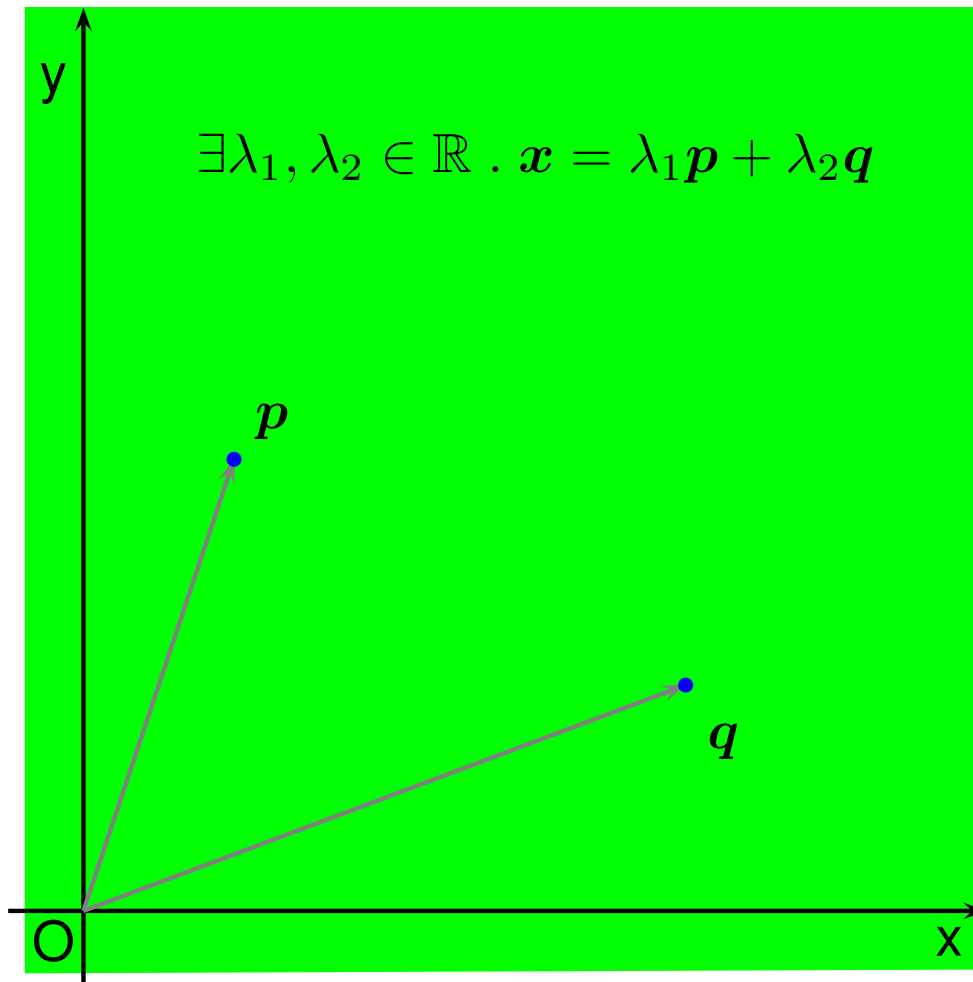
THE DOMAIN $\mathbb{CP}_n$ OF CLOSED CONVEX POLYHEDRA

Generator Representation: $\mathcal{P} = \text{gen}(\mathcal{G})$

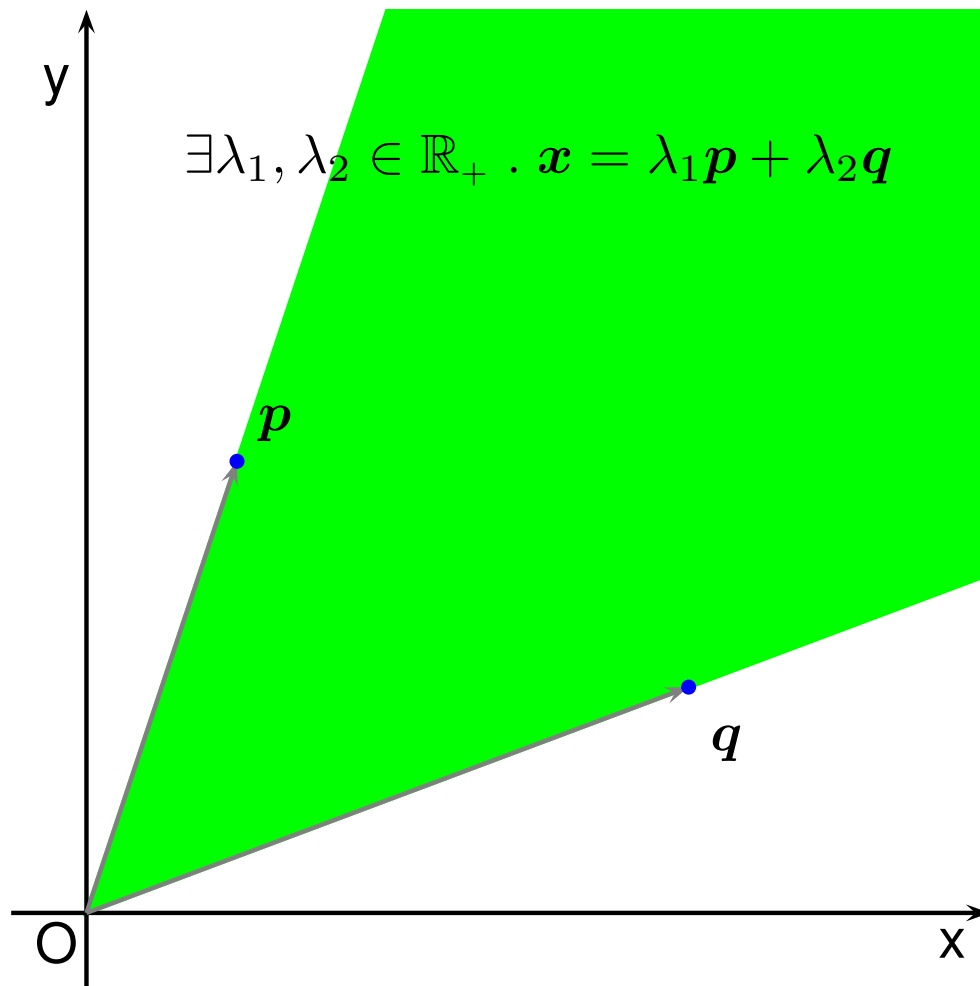
- $\mathcal{G} = (L, R, P)$, where
 - P is a finite set of **points** of $\mathcal{P}$;
 - R is a finite set of **rays** (directions of infinity) of $\mathcal{P}$;
 - L is a finite set of **lines** (bidirectional rays) of $\mathcal{P}$.
- No redundant generator + max number of lines $\implies$ **minimal form**.
- Weak notions of **canonical form** (e.g., by orthogonality).

$$\text{gen}(\mathcal{G}) \stackrel{\text{def}}{=} \left\{ L\lambda + R\rho + P\pi \in \mathbb{R}^n \mid \begin{array}{l} \lambda \in \mathbb{R}^\ell, \rho \in \mathbb{R}_+^r, \\ \pi \in \mathbb{R}_+^p, \sum_{i=1}^p \pi_i = 1 \end{array} \right\}$$

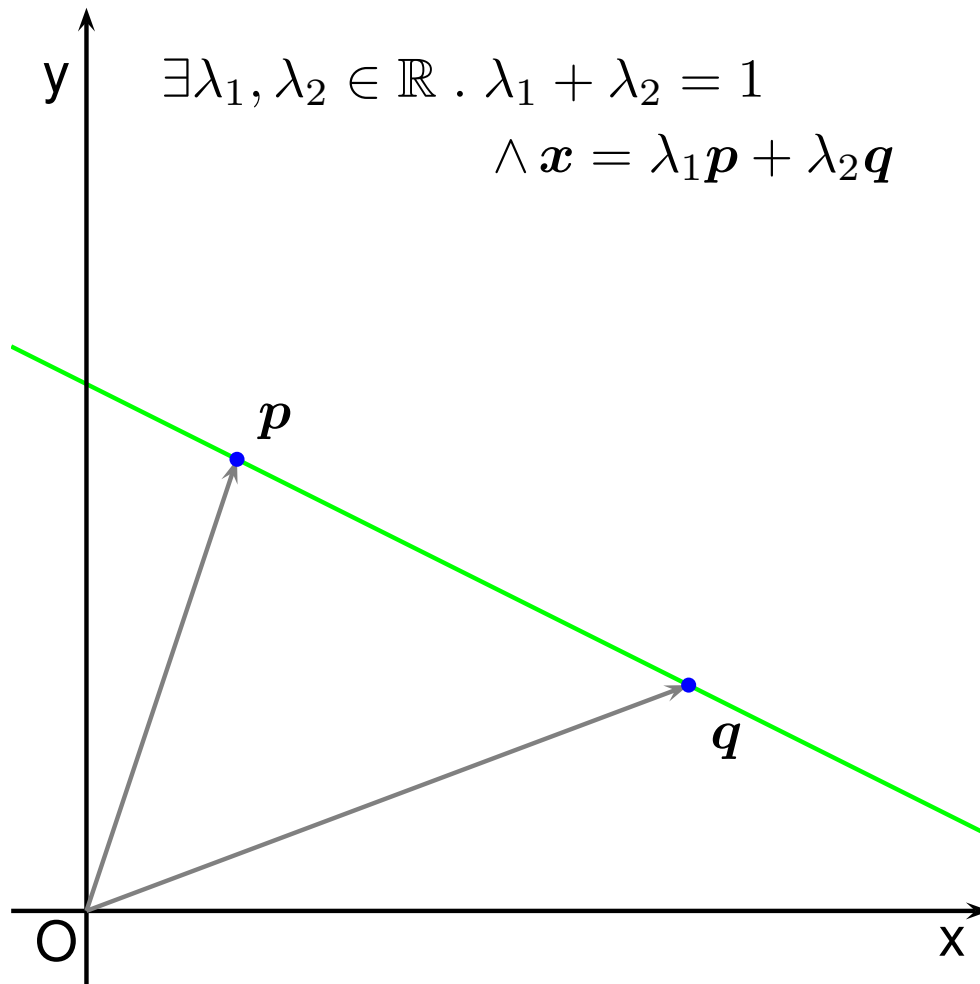
COMBINATIONS: LINEAR



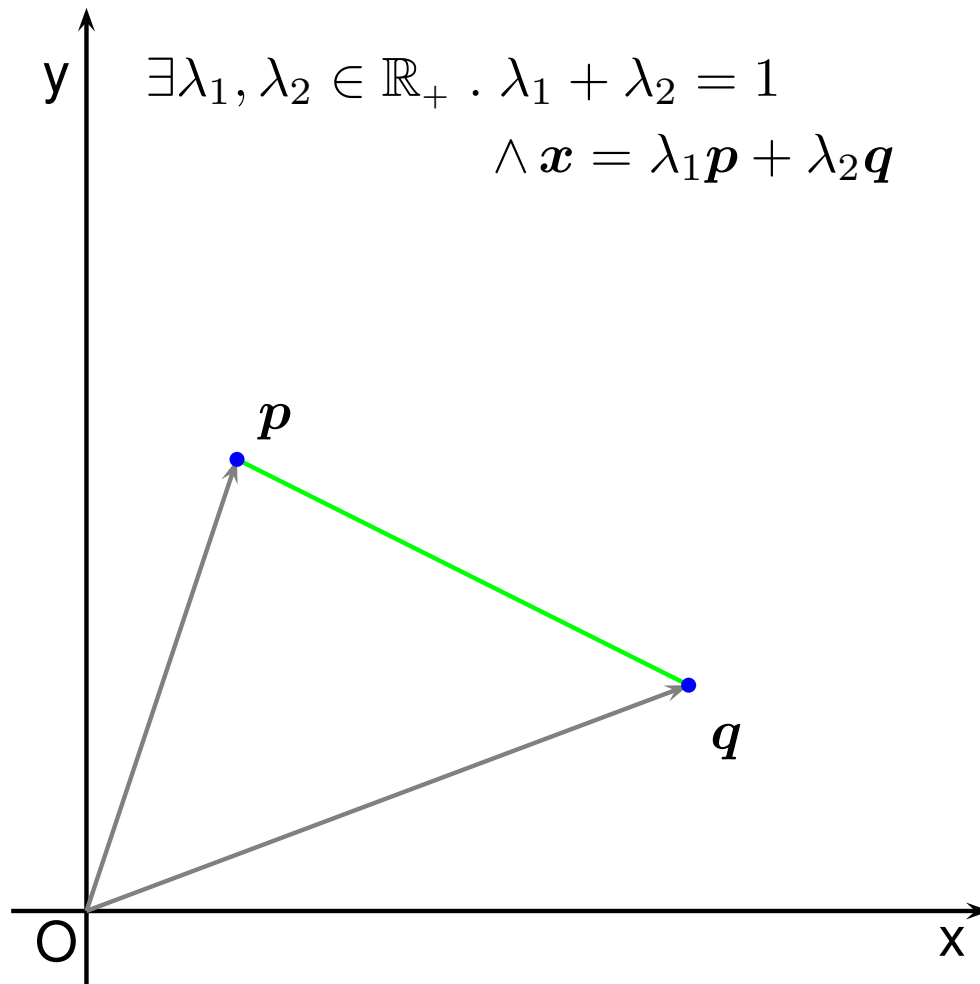
COMBINATIONS: CONIC



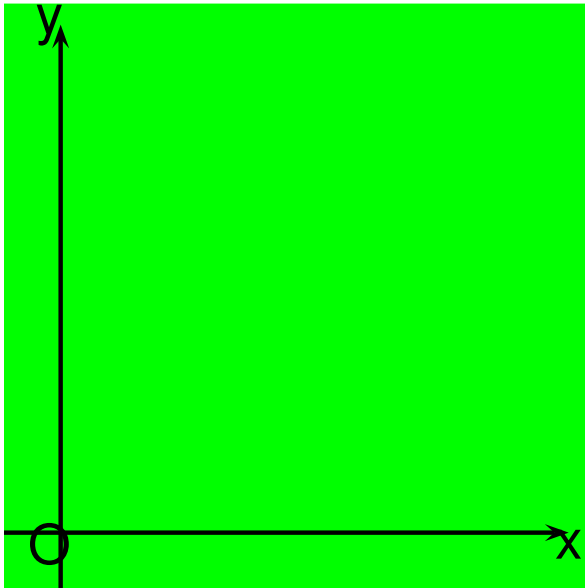
COMBINATIONS: AFFINE



COMBINATIONS: CONVEX (CONIC AND AFFINE)

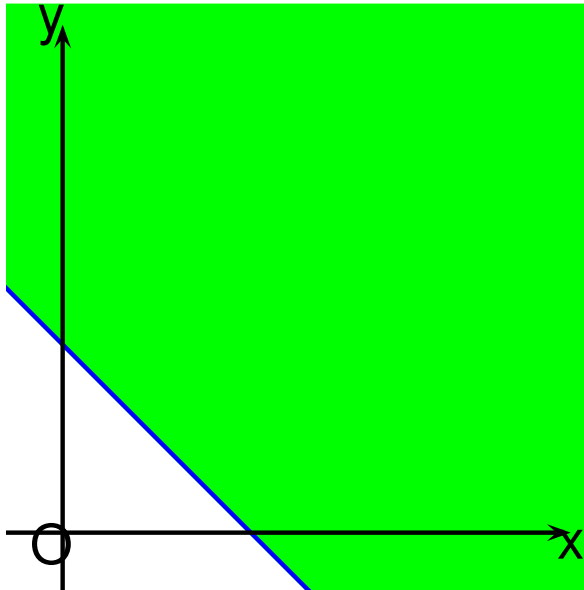


EXAMPLE: DOUBLE DESCRIPTION



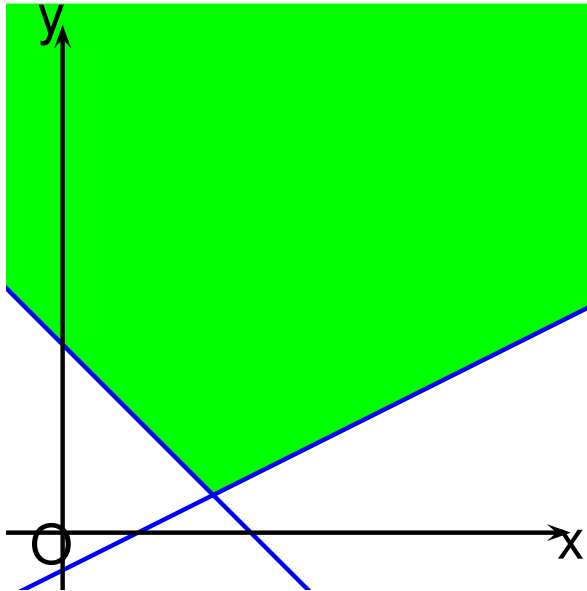
$$\begin{cases} x + y \geq 5 \\ x - 2y \leq 2 \\ y - 2x \leq 2 \end{cases}$$

EXAMPLE: DOUBLE DESCRIPTION



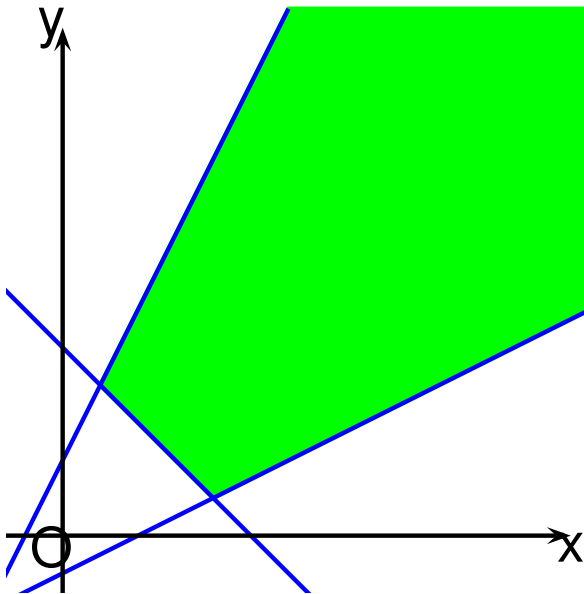
$$\begin{cases} x + y \geq 5 \\ x - 2y \leq 2 \\ y - 2x \leq 2 \end{cases}$$

EXAMPLE: DOUBLE DESCRIPTION



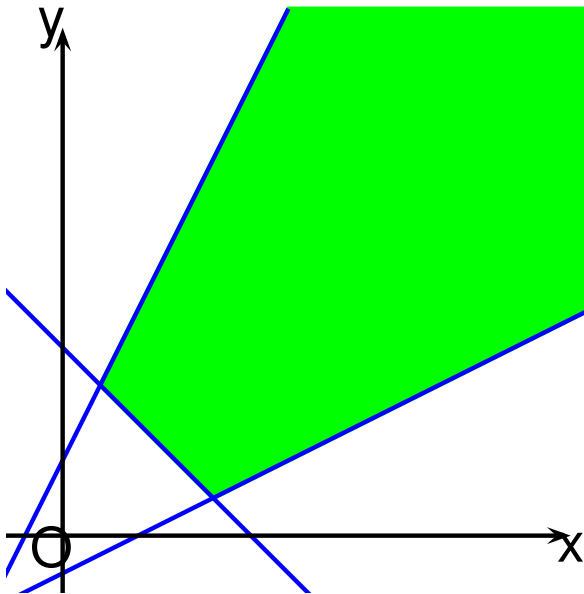
$$\begin{cases} x + y \geq 5 \\ x - 2y \leq 2 \\ y - 2x \leq 2 \end{cases}$$

EXAMPLE: DOUBLE DESCRIPTION

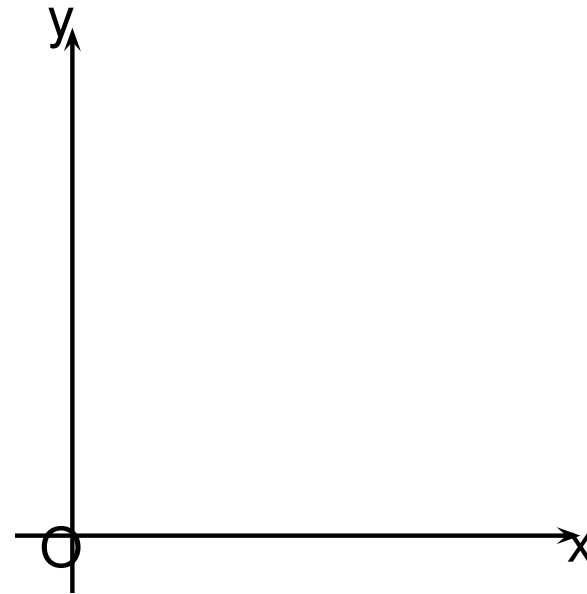


$$\begin{cases} x + y \geq 5 \\ x - 2y \leq 2 \\ y - 2x \leq 2 \end{cases}$$

EXAMPLE: DOUBLE DESCRIPTION

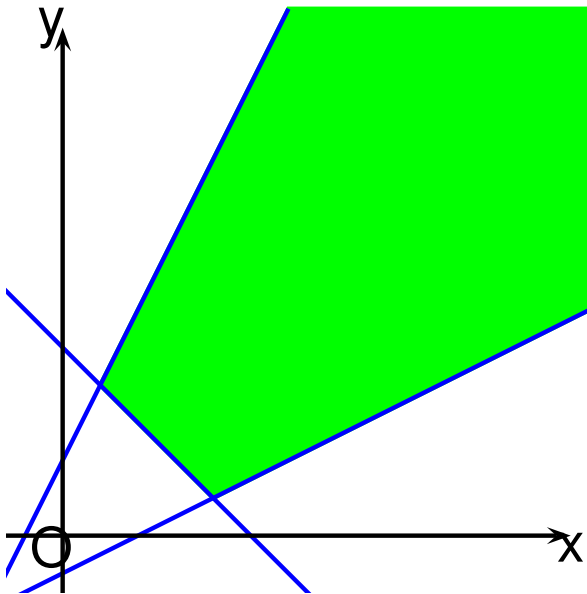


$$\begin{cases} x + y \geq 5 \\ x - 2y \leq 2 \\ y - 2x \leq 2 \end{cases}$$

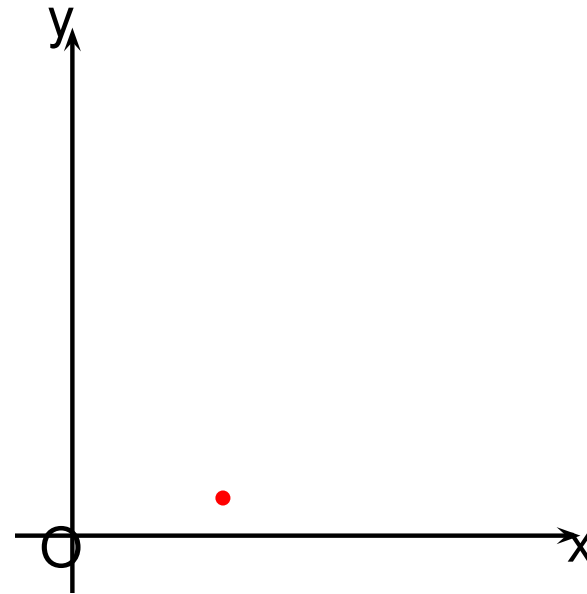


$$\begin{cases} \text{lines: } \emptyset \\ \text{points: } \emptyset \\ \text{rays: } \emptyset \end{cases}$$

EXAMPLE: DOUBLE DESCRIPTION

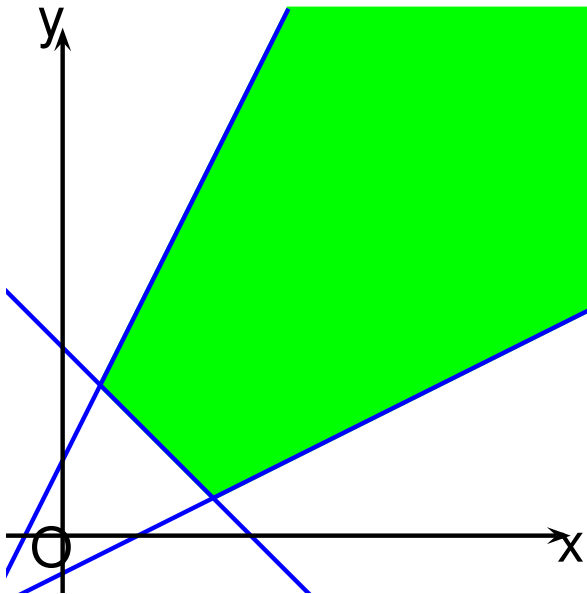


$$\begin{cases} x + y \geq 5 \\ x - 2y \leq 2 \\ y - 2x \leq 2 \end{cases}$$

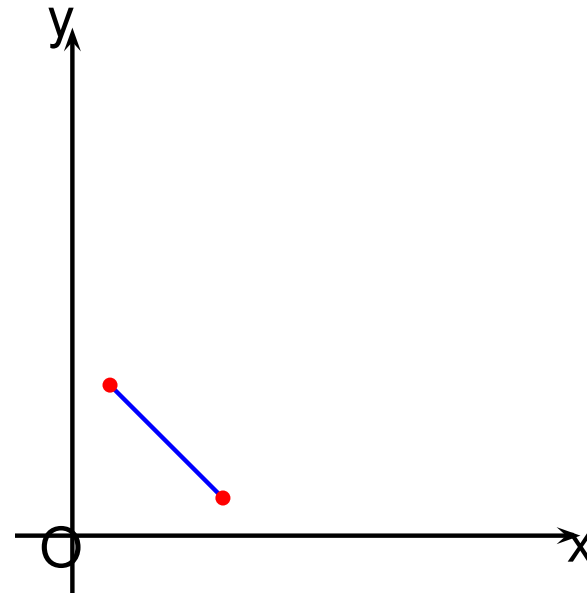


$$\begin{cases} \text{lines: } \emptyset \\ \text{points: } \{(4, 1)\} \\ \text{rays: } \emptyset \end{cases}$$

EXAMPLE: DOUBLE DESCRIPTION

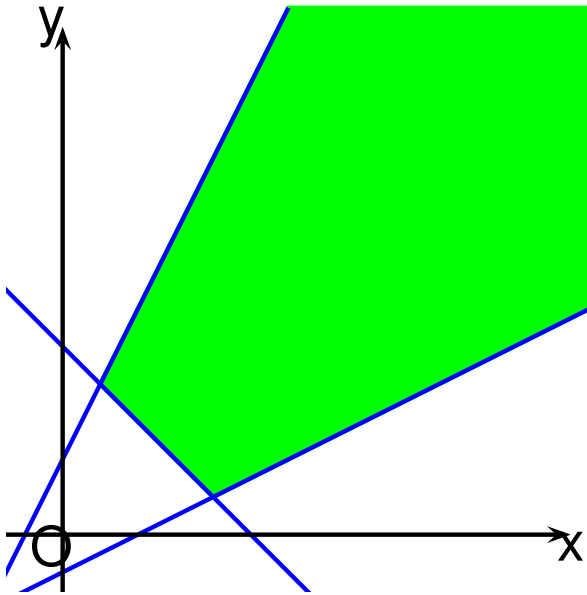


$$\begin{cases} x + y \geq 5 \\ x - 2y \leq 2 \\ y - 2x \leq 2 \end{cases}$$

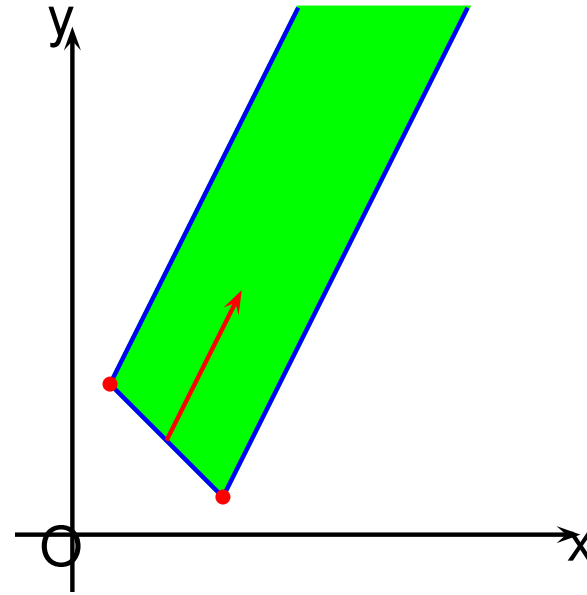


$$\begin{cases} \text{lines: } \emptyset \\ \text{points: } \{(4, 1), (1, 4)\} \\ \text{rays: } \emptyset \end{cases}$$

EXAMPLE: DOUBLE DESCRIPTION

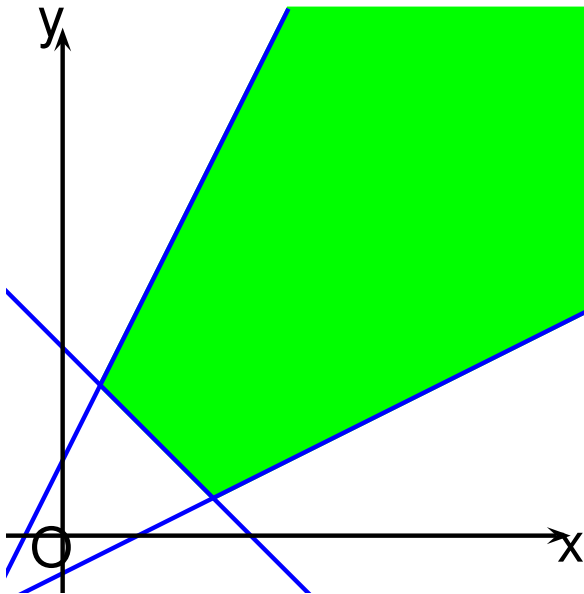


$$\begin{cases} x + y \geq 5 \\ x - 2y \leq 2 \\ y - 2x \leq 2 \end{cases}$$

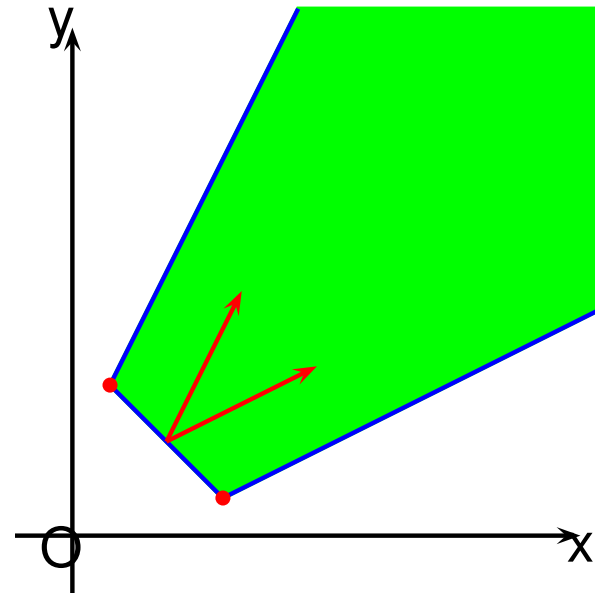


$$\begin{cases} \text{lines: } \emptyset \\ \text{points: } \{(4, 1), (1, 4)\} \\ \text{rays: } \{(1, 2)\} \end{cases}$$

EXAMPLE: DOUBLE DESCRIPTION



$$\begin{cases} x + y \geq 5 \\ x - 2y \leq 2 \\ y - 2x \leq 2 \end{cases}$$



$$\begin{cases} \text{lines: } \emptyset \\ \text{points: } \{(4, 1), (1, 4)\} \\ \text{rays: } \{(1, 2), (2, 1)\} \end{cases}$$

THE DD METHOD BY MOTZKIN ET AL.

The Principle of Duality

- Systems of constraints and generators enjoy a **duality property**.
- Very roughly speaking:
 - the constraints of a polyhedron are (almost) the generators of the *polar* of the polyhedron;
 - the generators of a polyhedron are (almost) the constraints of the polar of the polyhedron;
- ➡ Computing constraints from generators is the same problem as computing generators from constraints.

The Algorithm of Motzkin-Chernikova-Le Verge

- Solves both problems yielding a minimized DD pair.

But, wait a minute...

... why keeping two representations for the same object?

ADVANTAGES OF THE DUAL DESCRIPTION METHOD

Some operations are more efficiently performed on constraints

- Intersection is implemented as the union of constraint systems.
- Adding constraints (of course).
- Relation polyhedron-generator (subsumes or not).

Some operations are more efficiently performed on generators

- Convex polyhedral hull (poly-hull): union of generator systems.
- Adding generators (of course).
- Projection (i.e., removing dimensions).
- Relation polyhedron-constraint (disjoint, intersects, includes ...).
- Finiteness (boundedness) check.
- Time-elapse.

Some operations are more efficiently performed with both

- Inclusion and equality tests.
- Widening.

EXAMPLE: THE INCLUSION TEST

- Let $\mathcal{P}_1 = \text{gen}(\mathcal{G}_1) \in \mathbb{CP}_n$ and $\mathcal{P}_2 = \text{con}(\mathcal{C}_2) \in \mathbb{CP}_n$.
- $\mathcal{P}_1 \subseteq \mathcal{P}_2$ iff each generator in $\mathcal{G}_1$ satisfies each constraint in $\mathcal{C}_2$;
 - generator $g \in \mathbb{R}^n$ satisfies constraint $\langle a, x \rangle \bowtie b$ if and only if the scalar product $s \stackrel{\text{def}}{=} \langle a, g \rangle$ satisfies the following condition:

Generator type	Constraint type	
	$=$	$\geq$
line	$s = 0$	$s = 0$
ray	$s = 0$	$s \geq 0$
point	$s = b$	$s \geq b$

PROBLEMS IN THE ABSTRACT SEMANTICS COMPUTATION

- The “limit” of the abstract computation may not be representable in the abstract domain (e.g., a circle is not a polyhedron);
- Reaching a post-fixpoint of the abstract semantic function may require an infinite number of computation steps;
- Even when the abstract computation is intrinsically finite, it may be practically unfeasible if it requires too many abstract iterations; for instance,

```
x := 0;  
while (x < 1000) do  
  x := x+1; y := f(x);  
endwhile
```

Widening operators try to solve all of these problems at once.

DEFINITION OF WIDENING OPERATOR

A variant of the classical one (see Cousot and Cousot, PLILP'92):

- Let $\langle L, \sqsubseteq, \perp, \sqcup \rangle$ be a join-semi-lattice. Then, the operator $\nabla: L \times L \rightarrow L$ is a **widening** on L if
- ① $\forall x, y \in L : x \sqsubseteq y \implies y \sqsubseteq x \nabla y$;
 - ② for all increasing chains $y_0 \sqsubseteq y_1 \sqsubseteq \dots$, the chain defined by $x_0 \stackrel{\text{def}}{=} y_0, \dots, x_{i+1} \stackrel{\text{def}}{=} x_i \nabla (x_i \sqcup y_{i+1}), \dots$ is not strictly increasing.
- The upward iteration sequence with widenings (starting from $x_0 = \perp$)

$$x_{i+1} = \begin{cases} x_i, & \text{if } \mathcal{F}^\#(x_i) \sqsubseteq x_i; \\ x_i \nabla (x_i \sqcup \mathcal{F}^\#(x_i)), & \text{otherwise;} \end{cases}$$

converges (to a post-fixpoint of $\mathcal{F}^\#$) after a **finite number of iterations**.

∇ -COMPATIBLE LIMITED GROWTH ORDERING

- Any widening ∇ induces on L a partial order relation $\sqsubseteq_{\nabla}$ satisfying the **ascending chain condition** (ACC); this is the reflexive and transitive closure of

$$\{ (x, z) \in L \times L \mid \exists y \in L . x \sqsubset y \wedge z = x \nabla y \}.$$

- A **limited growth ordering** (lgo) is the strict version of a **finitely computable** preorder relation that satisfies the ACC on L .
- Let ∇ be a widening on L . An lgo $\curvearrowright$ is **∇ -compatible** if

$$\forall x, y \in L : x \sqsubset y \implies x \curvearrowright x \nabla y.$$

- A ∇ -compatible lgo is a **finite convergence certificate** for ∇ .

A FRAMEWORK FOR IMPROVING UPON A FIXED WIDENING

Suppose that

- ① $\nabla: L \times L \rightarrow L$ is a widening on the join-semi-lattice $\langle L, \sqsubseteq, \perp, \sqcup \rangle$;
- ② $\curvearrowright \subseteq L \times L$ is a ∇ -compatible lgo;
- ③ $h: L \times L \rightarrow L$ is an upper bound operator.

For all $x, y \in L$ such that $x \sqsubseteq y$, define

$$x \tilde{\nabla} y \stackrel{\text{def}}{=} \begin{cases} h(x, y), & \text{if } x \curvearrowright h(x, y) \sqsubseteq x \nabla y; \\ x \nabla y, & \text{otherwise.} \end{cases}$$

→ Then $\tilde{\nabla}$ is a widening operator at least as precise as ∇ .

NEW WIDENINGS FOR CONVEX POLYHEDRA

- In [Bagnara et al., SAS'03] we have instantiated the framework on the domain $\mathbb{CP}_n$, improving upon the standard widening.
- We have defined a fine-grained, **standard widening**-compatible lgo and used **four different heuristics**: do not widen, combining constraints, evolving points, evolving rays.
- Experiments have shown that the new widening significantly improves upon the precision of the standard widening.
- In general, this does **not** hold for the **final result of upward iteration sequences**, because neither the standard widening nor the new one are **monotonic operators**.
- Depending on the considered application, efficiency can be degraded. Several trade-off's are possible.

NNC POLYHEDRA

Strict Inequalities and NNC Polyhedra

- If $a \in \mathbb{R}^n$, $a \neq 0$, and $b \in \mathbb{R}$, the linear **strict** inequality constraint $\langle a, x \rangle > b$ defines an **open** affine half-space;
- when strict inequalities are allowed in the system of constraints we have polyhedra that are not necessarily closed: **NNC polyhedra**.

Encoding NNC Polyhedra as C Polyhedra

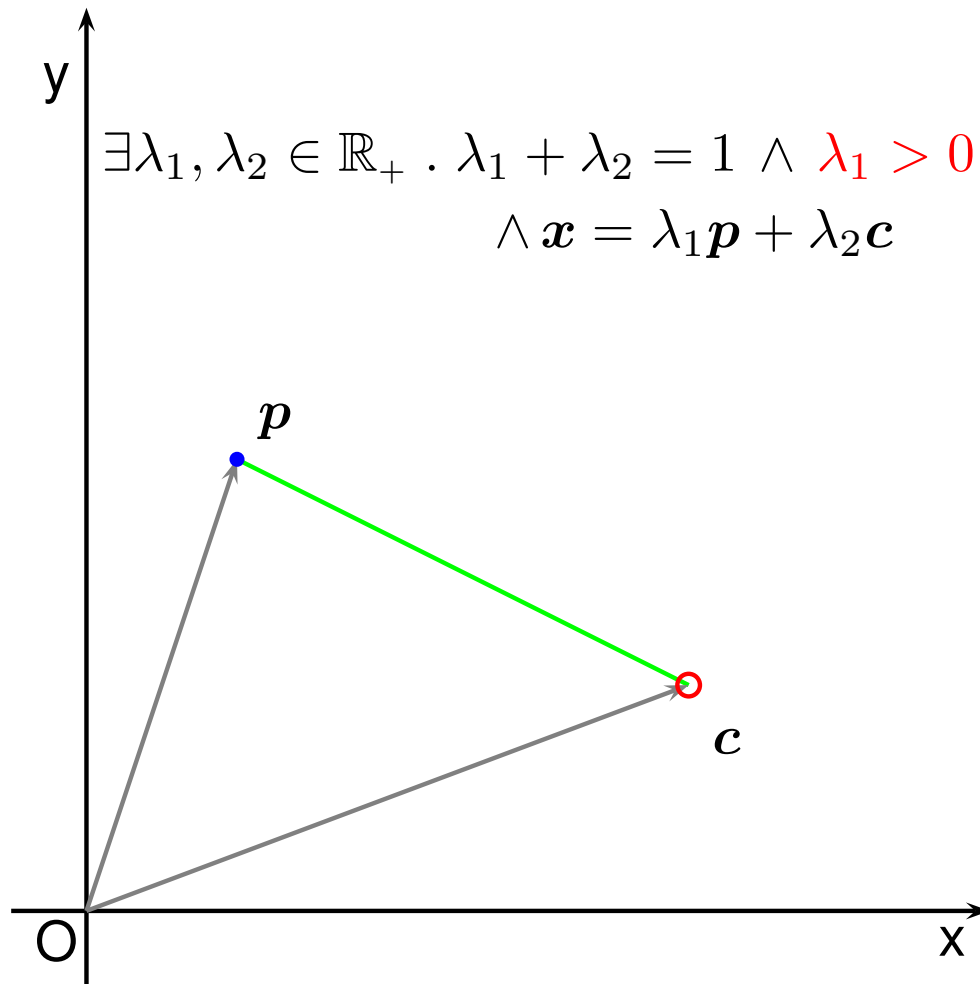
- call $\mathbb{P}_n$ and $\mathbb{CP}_n$ the sets of all NNC and closed polyhedra, respectively;
- each NNC polyhedron $\mathcal{P} \in \mathbb{P}_n$ can be embedded into a closed polyhedron $\mathcal{R} \in \mathbb{CP}_{n+1}$:
- the additional dimension of the vector space, usually labeled by the letter ϵ , encodes the topological closedness of each affine half-space in the constraint description for $\mathcal{P}$.

WHAT ARE THE GENERATORS OF NNC POLYHEDRA?

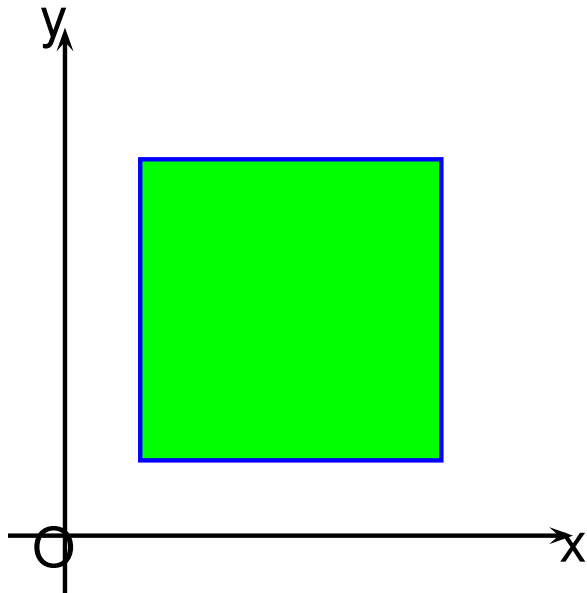
- A fundamental feature of the DD method: the ability to represent polyhedra both by constraints and generators.
- Previous works/implementations did not offer a satisfactory answer.
- An **intuitive generalization** was provided in [Bagnara et al., SAS'02], based on the introduction of a new kind of generators: **closure points**.
- Extended generator systems: $\mathcal{G} = (L, R, P, C)$.

$$\text{gen}(\mathcal{G}) \stackrel{\text{def}}{=} \left\{ L\lambda + R\rho + P\pi + C\gamma \mid \begin{array}{l} \lambda \in \mathbb{R}^\ell, \rho \in \mathbb{R}_+^r, \pi \in \mathbb{R}_+^p, \gamma \in \mathbb{R}_+^c, \\ \sum_{i=1}^p \pi_i + \sum_{i=1}^c \gamma_i = 1, \pi \neq \mathbf{0} \end{array} \right\}$$

COMBINATIONS: NNC CONVEX

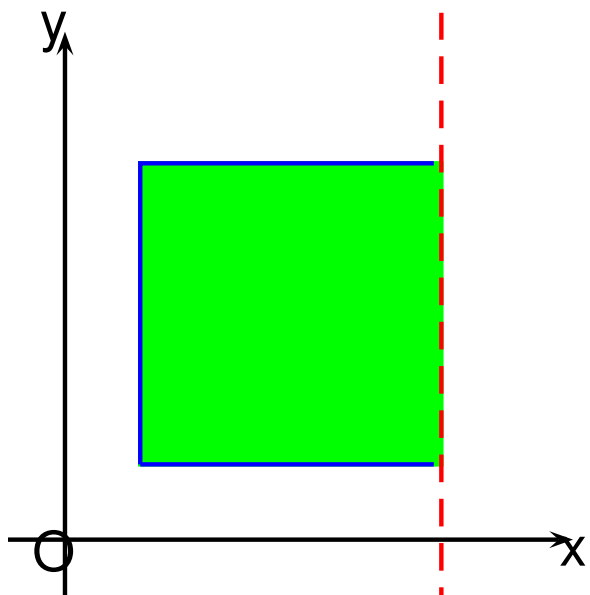


EXAMPLE: NNC DOUBLE DESCRIPTION



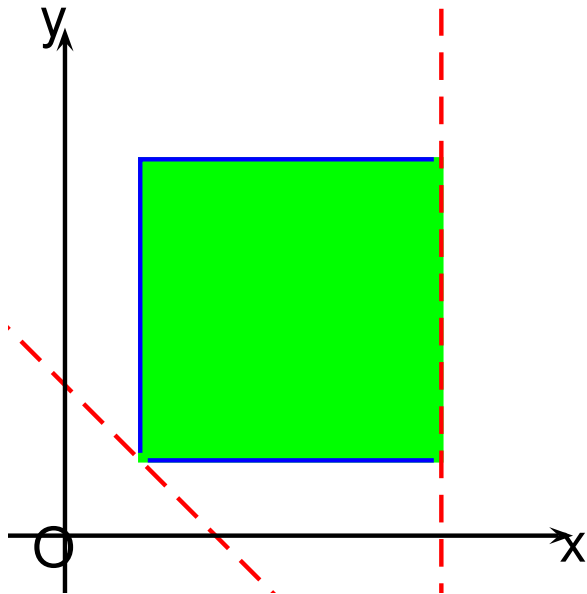
$$\begin{cases} 2 \leq x \leq 10 \\ 2 \leq y \leq 10 \end{cases}$$

EXAMPLE: NNC DOUBLE DESCRIPTION



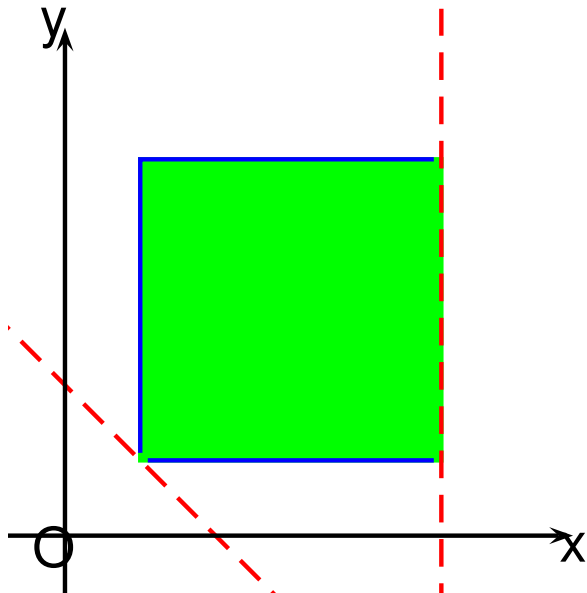
$$\begin{cases} 2 \leq x < 10 \\ 2 \leq y \leq 10 \end{cases}$$

EXAMPLE: NNC DOUBLE DESCRIPTION

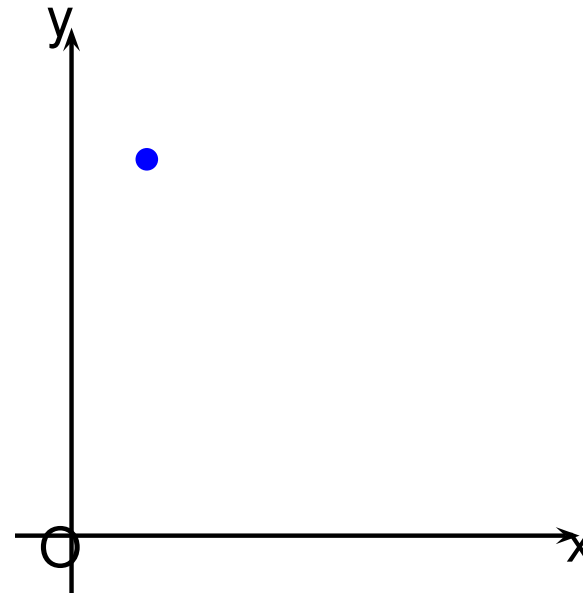


$$\begin{cases} 2 \leq x < 10 \\ 2 \leq y \leq 10 \\ x + y > 4 \end{cases}$$

EXAMPLE: NNC DOUBLE DESCRIPTION

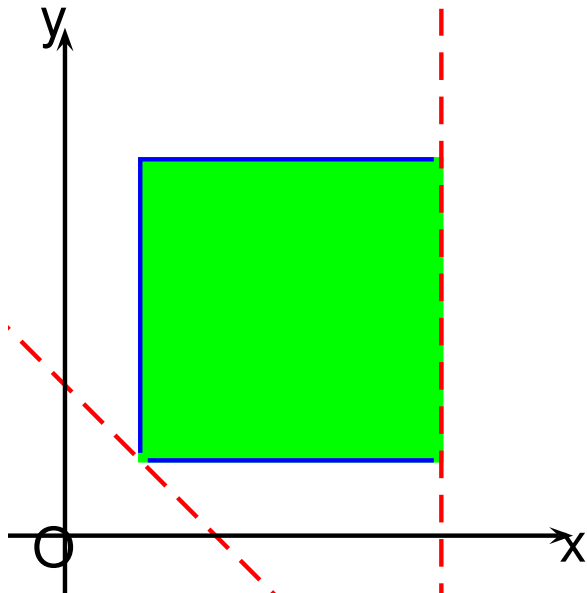


$$\begin{cases} 2 \leq x < 10 \\ 2 \leq y \leq 10 \\ x + y > 4 \end{cases}$$

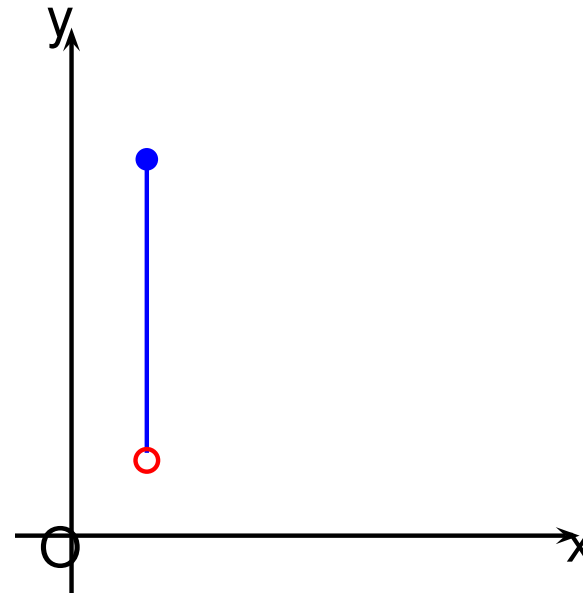


$$\begin{cases} \text{lines: } \emptyset & \text{rays: } \emptyset \\ \text{points: } \{(2, 10)\} \\ \text{c.p.: } \emptyset \end{cases}$$

EXAMPLE: NNC DOUBLE DESCRIPTION

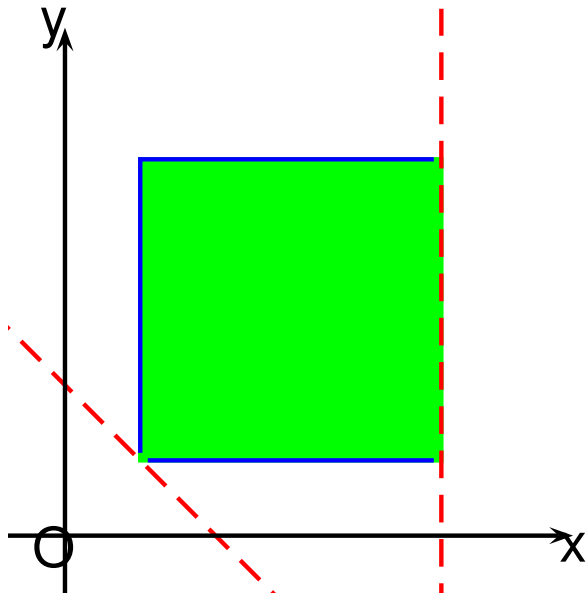


$$\begin{cases} 2 \leq x < 10 \\ 2 \leq y \leq 10 \\ x + y > 4 \end{cases}$$

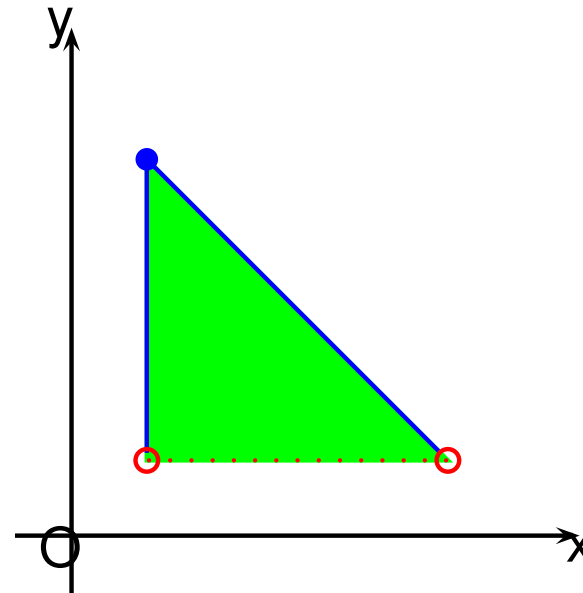


$$\begin{cases} \text{lines: } \emptyset & \text{rays: } \emptyset \\ \text{points: } \{(2, 10)\} \\ \text{c.p.: } \{(2, 2)\} \end{cases}$$

EXAMPLE: NNC DOUBLE DESCRIPTION

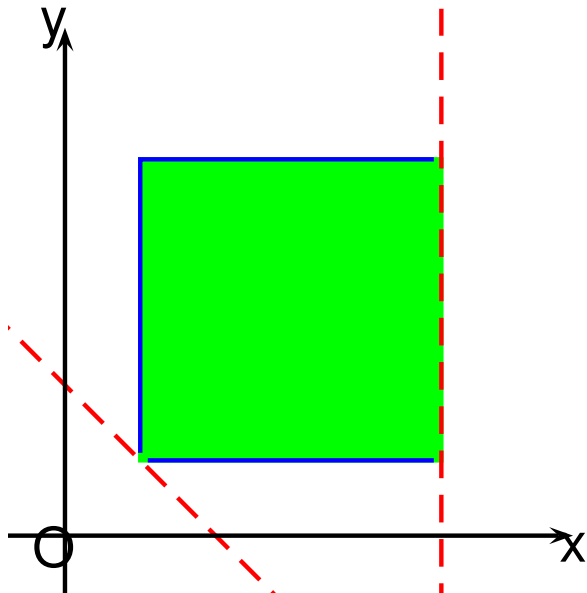


$$\begin{cases} 2 \leq x < 10 \\ 2 \leq y \leq 10 \\ x + y > 4 \end{cases}$$

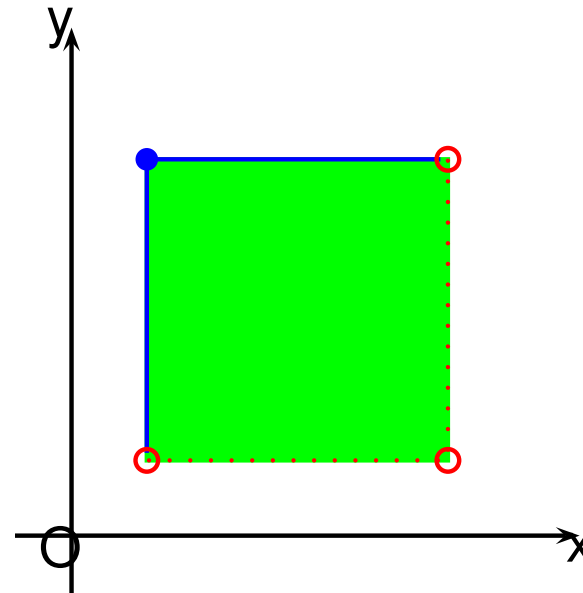


$$\begin{cases} \text{lines: } \emptyset & \text{rays: } \emptyset \\ \text{points: } \{(2, 10)\} \\ \text{c.p.: } \{(2, 2), (10, 2)\} \end{cases}$$

EXAMPLE: NNC DOUBLE DESCRIPTION

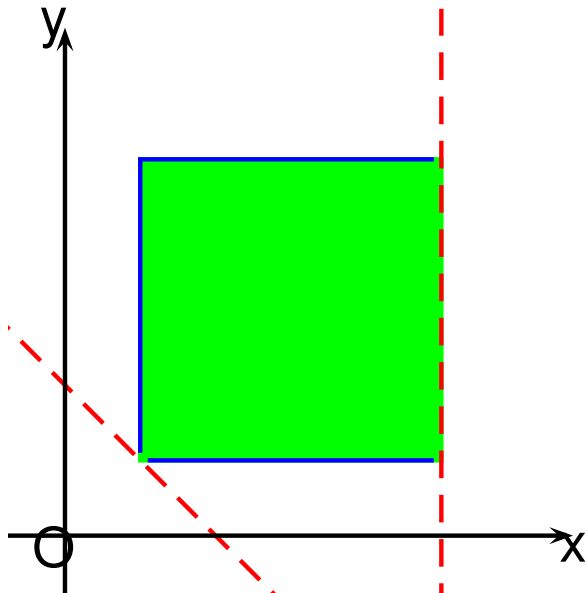


$$\begin{cases} 2 \leq x < 10 \\ 2 \leq y \leq 10 \\ x + y > 4 \end{cases}$$

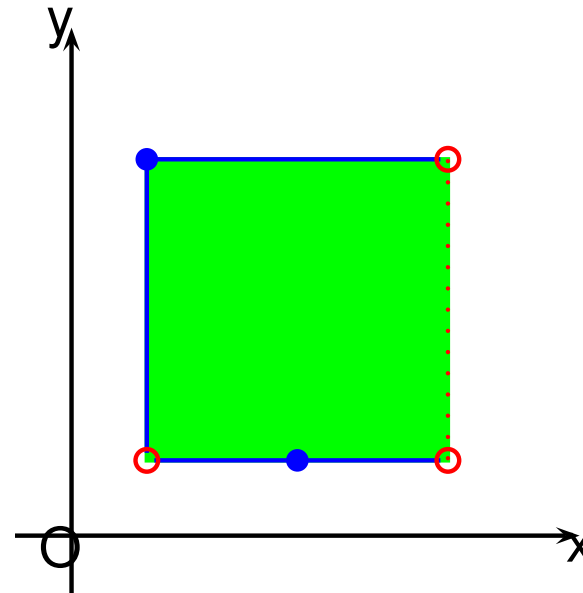


$$\begin{cases} \text{lines: } \emptyset & \text{rays: } \emptyset \\ \text{points: } \{(2, 10)\} \\ \text{c.p.: } \{(2, 2), (10, 2), (10, 10)\} \end{cases}$$

EXAMPLE: NNC DOUBLE DESCRIPTION



$$\begin{cases} 2 \leq x < 10 \\ 2 \leq y \leq 10 \\ x + y > 4 \end{cases}$$



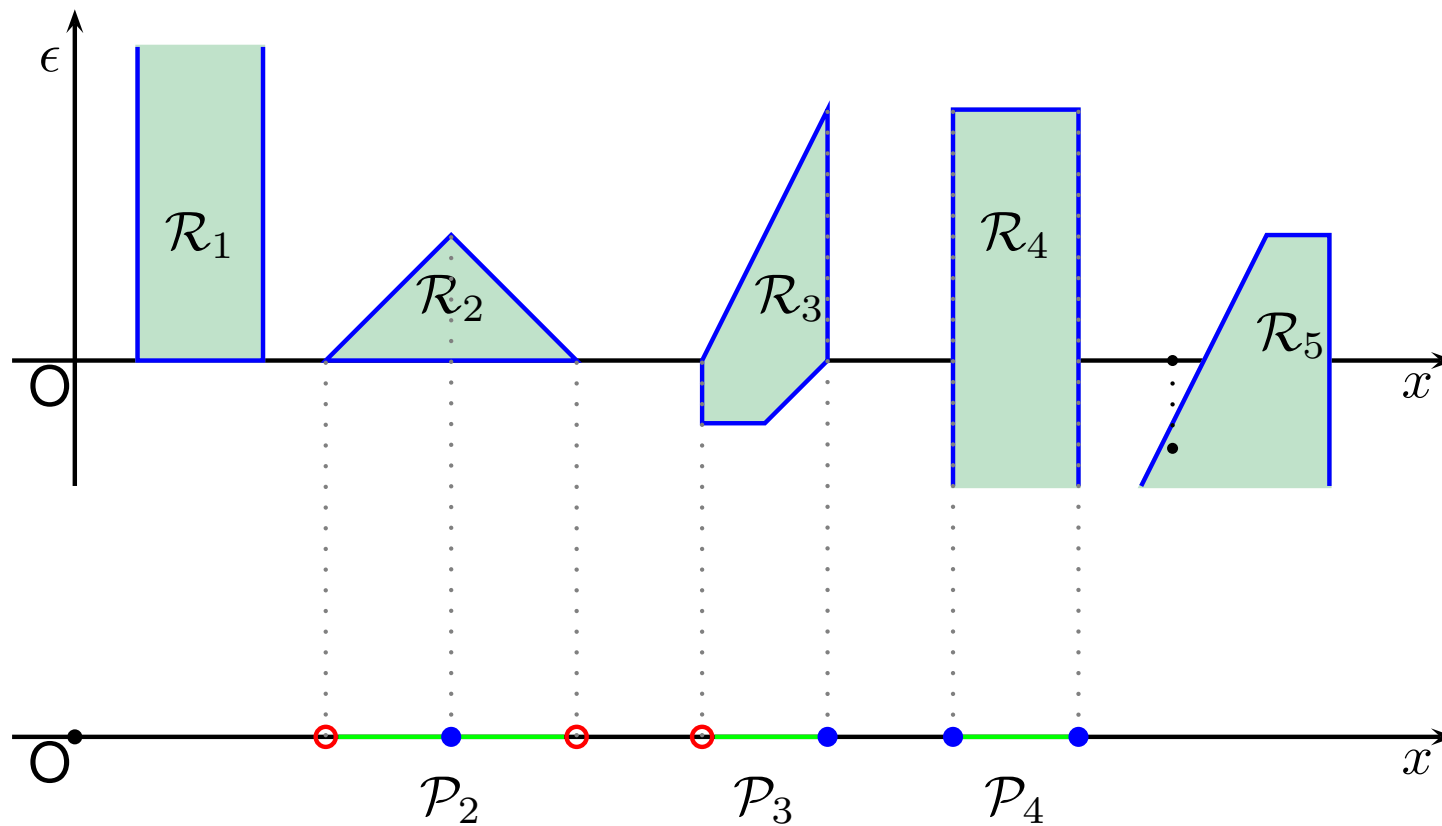
$$\begin{cases} \text{lines: } \emptyset & \text{rays: } \emptyset \\ \text{points: } \{(2, 10), (6, 2)\} \\ \text{c.p.: } \{(2, 2), (10, 2), (10, 10)\} \end{cases}$$

EXAMPLE: THE INCLUSION TEST FOR NNC POLYHEDRA

- Let $\mathcal{P}_1 = \text{gen}(\mathcal{G}_1) \in \mathbb{P}_n$ and $\mathcal{P}_2 = \text{con}(\mathcal{C}_2) \in \mathbb{P}_n$.
- $\mathcal{P}_1 \subseteq \mathcal{P}_2$ iff each generator in $\mathcal{G}_1$ **satisfies** each constraint in $\mathcal{C}_2$.
- Generator $g \in \mathbb{R}^n$ satisfies constraint $\langle a, x \rangle \bowtie b$ if and only if the scalar product $s \stackrel{\text{def}}{=} \langle a, g \rangle$ satisfies the following condition:

Generator type	Constraint type		
	=	$\geq$	$>$
line	$s = 0$	$s = 0$	$s = 0$
ray	$s = 0$	$s \geq 0$	$s \geq 0$
point	$s = b$	$s \geq b$	$s > b$
closure point	$s = b$	$s \geq b$	$s \geq b$

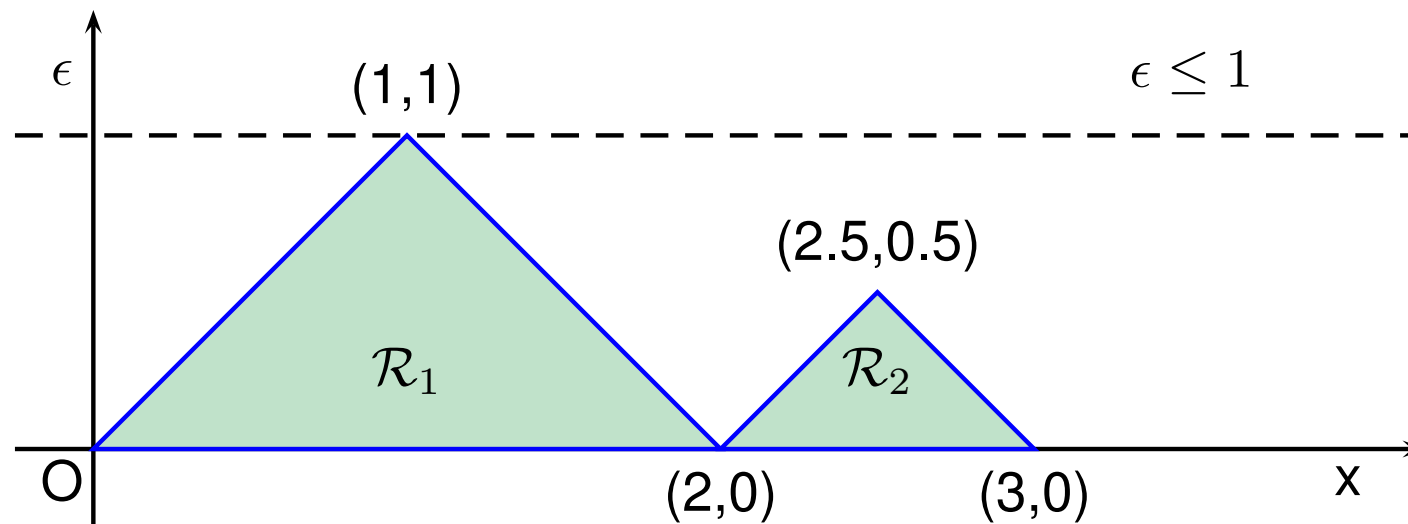
NNC IMPLEMENTATION: ϵ -POLYHEDRA



NNC IMPLEMENTATION: A MINIMIZATION ISSUE

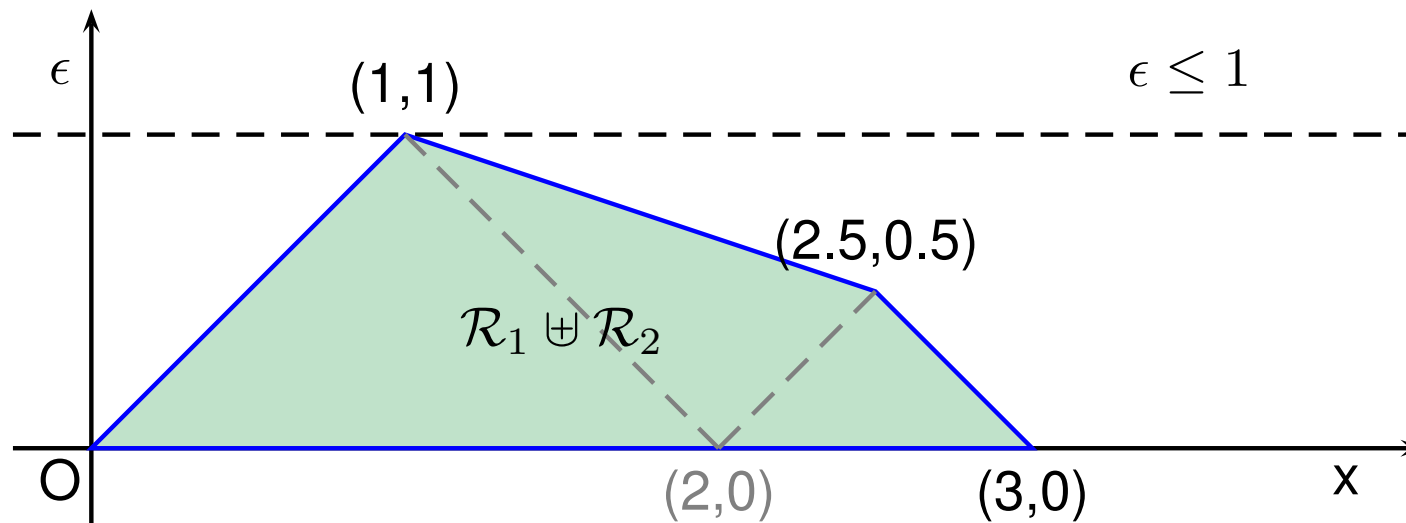
$\mathcal{R}_1$ encodes $\mathcal{P}_1 = \text{con}(\{0 < x < 2\})$,

$\mathcal{R}_2$ encodes $\mathcal{P}_2 = \text{con}(\{2 < x < 3\})$.

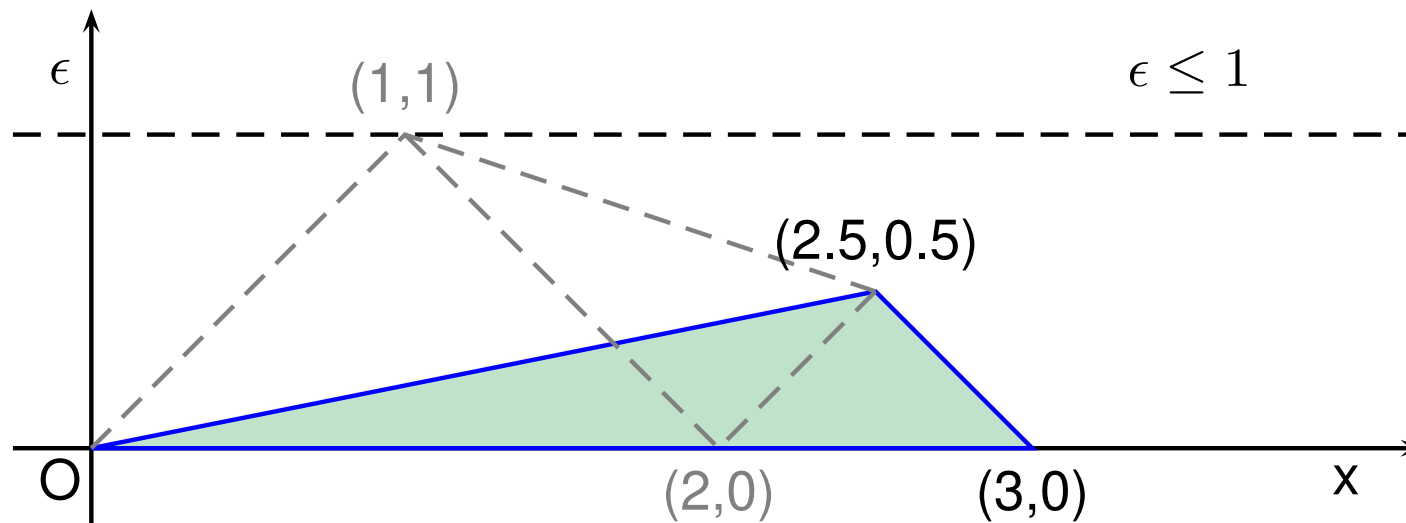


NNC IMPLEMENTATION: A MINIMIZATION ISSUE

$\mathcal{R}_1 \uplus \mathcal{R}_2$ encodes the poly-hull $\mathcal{P}_1 \uplus \mathcal{P}_2 = \text{con}(\{0 < x < 3\})$, but it also encodes the **redundant constraint** $x < 4$.



NNC IMPLEMENTATION: ϵ -MINIMAL FORMS!



THE PARMA POLYHEDRA LIBRARY

- A collaborative project started in January 2001 at the Department of Mathematics of the **University of Parma**.
 - The **University of Leeds** (UK) is now a major contributor to the library.
- It aims at becoming a **truly professional library** for the handling of a wide range of **numerical abstractions** targeted at abstract interpretation and computer-aided verification.
 - Currently provides support for (not necessarily closed) convex polyhedra and finite sets of (NNC) polyhedra.
- **Free software** released under the GNU General Public License.

PPL FEATURES

Portability across different computing platforms

- written in standard C++;
- but the the client application needs not be written in C++.

Absence of arbitrary limits

- arbitrary precision integer arithmetic for coefficients and coordinates;
- all data structures can expand automatically (in amortized constant time) to any dimension allowed by the available virtual memory.

Complete information hiding

- the internal representation of constraints, generators and systems thereof need not concern the client application;
- implementation devices such as the *positivity constraint* or ϵ -polyhedra are invisible from outside.

PPL FEATURES: HIDING PAYS

Expressivity

- ' $X + 2*Y + 5 \geq 7*Z$ ' and ' $\text{ray}(3*X + Y)$ ' is valid syntax both for the C++ and the Prolog interfaces;
- the Objective Caml and Java interfaces are equally friendly;
- even the C interface refers to concepts like linear expression, constraint and constraint system
 - (not to their possible implementations such as vectors and matrices).

Failure avoidance and detection

- illegal objects cannot be created easily;
- the interface invariants are systematically checked.

Efficiency

- can systematically apply incremental and lazy computation techniques.

PPL FEATURES: LAZINESS AND INCREMENTALITY

Dual description

- we may have a constraint system, a generator system, or both;
- in case only one is available, the other is recomputed only when it is convenient to do so.

Minimization

- the constraint (generator) system may or may not be minimized;
- it is minimized only when convenient.

Saturation matrices

- when both constraints and generators are available, some computations record here the relation between them for future use.

Sorting matrices

- for certain operations, it is advantageous to sort (lazily and incrementally) the matrices representing constraints and generators.

PPL FEATURES: SUPPORT FOR ROBUSTNESS

```
void complex_function(PH& ph1, const PH& ph2 ...) {  
    try {  
        start_timer(max_time_for_complex_function);  
        complex_function_on_polyhedra(ph1, ph2 ...);  
        stop_timer();  
    }  
    catch (Exception& e) { // Out of memory or timeout...  
        BoundingBox bb1, bb2;  
        ph1.shrink_bounding_box(bb1);  
        ph2.shrink_bounding_box(bb2);  
        complex_function_on_bounding_boxes(bb1, bb2 ...);  
        ph1 = Polyhedron(bb1);  
    }  
}
```

(KNOWN) USERS OF THE LIBRARY

- VERIMAG, FR (D. Merchat et al., Cartesian factoring)
- U. of Réunion, FR (F. Menard et al., cTI)
- Carnegie Mellon U., USA (K. Mixer et al., Action Language Verifier and G. Frehse, Linear Hybrid Automata)
- Delft U. of Technology, DK (M. Rhode)
- U. of Kent at Canterbury, UK (A. Simon, floating-point computations)
- ENS Cachan, FR (E. Fersman, model checking of hybrid systems)
- U. of Michigan, USA (H. Song, extending Spin to hybrid contexts)
- U. of Wisconsin, USA (D. Gopan et al., extending TVLA)
- Stanford U., USA (S. Sankaranarayanan et al., StInG)
- U. of Cambridge, UK (E. Upton et al., gated data dependence graphs)
- U. of Tel Aviv, IL (M. Sagiv et al., string cleanness for C programs)
- ...